

配列と関数

2026 年度高 2 情報

2026/05/23

配列

- 同じ種類のデータ（数値のみ、文字列のみ等）を順番に並べて保存できる大きな箱のようなものを**配列**という.
- 各データには **インデックス（番号）** を使ってアクセスできる.

注意

C#（や多くの言語）では、インデックスは **1** ではなく「**0**」から始まるが、1 から始まる言語もある. よって、入試などでは問題文の設定をよく確認する必要がある.

配列の宣言と使い方

```
public class ArrayEx{  
    public static void Main(){  
        int[] scores = new int[5];  
        // 1. 宣言とメモリの確保  
        scores[0] = 85;  
        scores[1] = 92;  
        int[] myScores = { 10, 20, 30, 40, 50 };  
        // 2. 宣言と同時に初期化  
        // 3. データへのアクセス  
        System.Console.WriteLine(myScores[0]);  
        // 結果: 10  
    }  
}
```

関数（メソッド）

特定の処理をひとまとめにして、名前をつけたもの

関数を使う3つのメリット：

- **再利用性**：一度書けば、どこからでも何度でも呼び出せる
- **可読性**：複雑な処理に「名前」がつくので、プログラムが読みやすくなる
- **保守性**：修正が必要な場合、関数の1箇所を直すだけで済む

関連する用語：

- **引数（ひきすう）**：関数に渡す入力データ
- **戻り値（もどりち）**：関数が処理を終えて返す結果

関数の基本構文

```
public class FunctionEx{  
    // 2つの数字を受け取り、合計を返す関数  
    public static int Add(int a, int b) {  
        return a + b; // 結果を返す  
    }  
    public static void Main(){  
        // 関数の呼び出し  
        int result = Add(10, 20);  
        System.Console.WriteLine(result); // 結果: 30  
    }  
}
```

配列と関数

```
public class ArrayAndFunction{
    // 配列内の数値をすべて足す関数
    public static int SumArray(int[] numbers){
        int total = 0;
        for (int i = 0; i < numbers.Length; i++) {
            total += numbers[i];
        }
        return total;
    }
    public static void Main(){
        int[] testScores = { 80, 90, 100 };
        System.Console.WriteLine(SumArray(testScores));
    }
    // 結果: 270
}
```

問題 1

問題 1

- n 円のものに m 円を支払ったとする (故に $m \geq n$ は仮定されているとしよう)
- 最小の枚数 (お札 + 硬貨の枚数) でお釣りを返すとき, その枚数を計算する関数をつくれ
- 使用する硬貨または紙幣は 10000 円札と 2000 円札を除いたものとする
- また, それが確実に動作することも実際の手計算をもって検証せよ

問題2

問題2

整数の配列を受け取り、その **平均値 (double 型)** を計算して返す関数 `CalculateAverage` を作成せよ.

要件：

- `for` 文と `values.Length` を使って、合計を計算.
- 割り算の結果を小数にするため、合計値を (`double`) でキャスト (型変換) する必要がある
- `int` 型 `sum` を `double` 型にするには, `(double)sum` とする.

問題 3

問題 3

整数の配列を受け取り，その中で一番大きい数値を見つけて返す関数を作成せよ．